

Kompilacja jądra systemowego Linux ze źródeł w środowisku wirtualnym

System Linux od podszewki

Kacper Bednarczuk

1. Specyfikacja środowiska

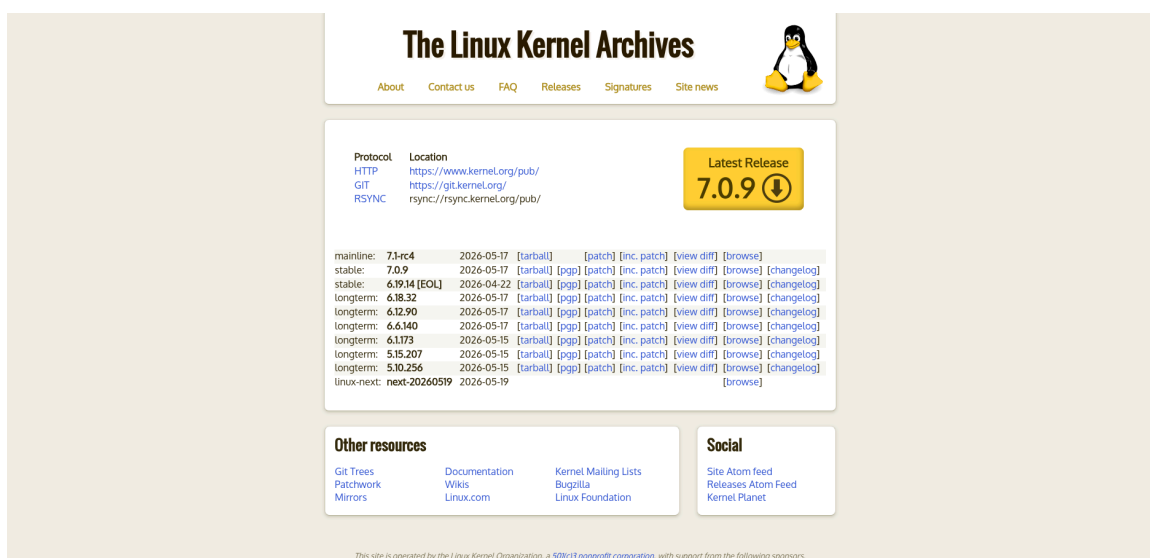
Podstawą realizacji zadania było przygotowanie wirtualnego środowiska uruchomieniowego za pomocą hipernadzorcy **KVM/QEMU** zarządzanego przez nakładkę graficzną `virt-manager`. Maszyna wirtualna została skonfigurowana z następującymi parametrami:

Parametr	Wartość
System operacyjny	Slackware 15.0 (x86_64), tryb tekstowy (bez środowiska graficznego)
Procesory wirtualne (vCPU)	8 (tryb <code>host-passthrough</code>)
Pamięć operacyjna (RAM)	8192 MiB (8 GiB)
Dysk	40 GiB, format <code>qcow2</code> (alokacja dynamiczna), magistrala VirtIO
Oprogramowanie układowe	UEFI (edk2-ovmf)
Chipset	i440FX
Karta sieciowa	VirtIO, tryb NAT (mostek <code>virbr0</code> , DHCP obsługiwany przez <code>dnsmasq</code>)
Bootloader	GRUB 2 (zainstalowany ręcznie w miejsce domyślnego ELILO)

Schemat partycjonowania dysku (/dev/vda , tablica GPT):

Partycja	Punkt montowania	Rozmiar	System plików	Przeznaczenie
/dev/vda1	/	26 GiB	ext4	Partycja główna (root)
/dev/vda2	/home	9.5 GiB	ext4	Dane użytkownika
/dev/vda3	swap	4 GiB	Linux Swap	Przestrzeń wymiany
/dev/vda4	/boot/efi	511 MiB	FAT32	Partycja EFI

Wersja jądra wybrana do kompilacji to 7.0.9, która w momencie realizacji tego laboratorium stanowiła najnowszą stabilną wersję dostępną w oficjalnych repozytoriach.



Strona główna kernel.org z wersją 7.0.9

2. Konfiguracja sieciowa - rozwiązywanie problemów

Bezpośrednio po instalacji systemu operacyjnego i próbie aktualizacji pakietów za pomocą `slackpkg` zidentyfikowano problem z brakiem dostępu do internetu wewnątrz maszyny wirtualnej. Klient DHCP (`dhcpcd`) na interfejsie `eth0` nie otrzymywał prawidłowej adresacji IPv4 od usługi `dnsmasq` , otrzymując w zamian adresy z puli APIPA (`169.254.x.x`). Komenda diagnostyczna `ping -c 3 google.com` kończyła się komunikatem o niemożności rozwiązania nazwy hosta (`Temporary failure in name resolution`).

Po weryfikacji konfiguracji po stronie systemu nadrzędnego ustalono, że ruch sieciowy na wirtualnym mostku `virbr0` był blokowany przez restrykcyjną politykę zapory UFW (Uncomplicated Firewall). Zapora nie posiadała reguł zezwalających na przepuszczanie pakietów DHCP (port 67 UDP) ani na routing pakietów pomiędzy interfejsem wirtualnym a siecią zewnętrzną.

W celu przywrócenia poprawnej komunikacji sieciowej w systemie nadrzędnym zaaplikowano następujące reguły:

```
sudo ufw allow in on virbr0
sudo ufw allow out on virbr0
sudo ufw route allow in on virbr0
sudo ufw route allow out on virbr0
```

Po wykonaniu powyższych instrukcji maszyna wirtualna bezzwłocznie otrzymała poprawne parametry konfiguracyjne z serwera DHCP (adres z puli 192.168.122.0/24), odzyskując łączność ze światem zewnętrznym.

3. Kompilacja: Metoda 1 (localmodconfig)

Pierwsza z zastosowanych metod optymalizacji polegała na wygenerowaniu konfiguracji na podstawie aktualnie załadowanych modułów w działającym systemie. Jest to podejście pozwalające na zredukowanie czasu kompilacji oraz rozmiaru wynikowego obrazu jądra poprzez eliminację sterowników i podsystemów, które nie są wymagane przez bieżący sprzęt.

3.1. Pobranie i rozpakowanie źródeł

Pobrano archiwum z kodem źródłowym jądra ze strony kernel.org i wypakowano je do katalogu /usr/src.

```
cd /usr/src
wget https://cdn.kernel.org/pub/linux/kernel/v7.x/linux-7.0.9.tar.xz
```

```
root@slack64:~# cd /usr/src
root@slack64:/usr/src# wget https://cdn.kernel.org/pub/linux/kernel/v7.x/linux-7.0.9.tar.xz
--2026-05-18 11:13:28-- https://cdn.kernel.org/pub/linux/kernel/v7.x/linux-7.0.9.tar.xz
Resolving cdn.kernel.org (cdn.kernel.org)... 146.75.117.176, 2a04:4e42:8d::432
Connecting to cdn.kernel.org (cdn.kernel.org)[146.75.117.176]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 157168936 (150M) [application/x-xz]
Saving to: 'linux-7.0.9.tar.xz'

linux-7.0.9.tar.xz      100%[=====] 149.89M  36.2MB/s   in 6.0s
2026-05-18 11:13:29 (24.8 MB/s) - 'linux-7.0.9.tar.xz' saved [157168936/157168936]
```

Pobieranie źródeł jądra

```
tar -xvpf linux-7.0.9.tar.xz
cd linux-7.0.9
```

```
linux-7.0.9/virt/kvm/vfio.h
linux-7.0.9/virt/lib/
linux-7.0.9/virt/lib/Kconfig
linux-7.0.9/virt/lib/Makefile
linux-7.0.9/virt/lib/irqbypass.c
root@slack64:/usr/src# cd linux-7.0.9
root@slack64:/usr/src/linux-7.0.9# ls
COPYING  Documentation/  Kconfig  MAINTAINERS  README  block/  crypto/  fs/  init/  ipc/  lib/  net/  samples/  security/  tools/  virt/
CREDITS  kbuild  LICENSES/  Makefile  arch/  certs/  drivers/  include/  io_uring/  kernel/  mm/  rust/  scripts/  sound/  usr/
```

Rozpakowywanie archiwum ze źródłami

3.2. Przygotowanie bazowej konfiguracji

Jako punkt wyjścia skopiowano konfigurację aktualnie działającego jądra systemowego.

```
zcat /proc/config.gz > .config
```

```
root@slack64:/usr/src/linux-7.0.9# zcat /proc/config.gz > .config
root@slack64:/usr/src/linux-7.0.9# ls -A
.config          .clang-format    .gitattributes   .pylintrc        CREDITS          Kconfig          Makefile         block/          drivers/         init/            kernel/          net/             scripts/         tools/
.cliippy.toml    .editorconfig    .gitignore       .rustfmt.toml    Documentation/   LICENSES/        README          certs/          fs/             io_uring/        lib/            rust/           security/        usr/
.coccinconfig    .get_maintainer.ignore .mailmap         COPYING          Kbuild           MAINTAINERS      arch/           crypto/         include/         ipc/            mm/             samples/         sound/          virt/
```

Kopiowanie konfiguracji bazowej

3.3. Generowanie zminimalizowanej konfiguracji

Komenda `make localmodconfig` porównuje listę aktualnie załadowanych modułów (uzyskaną z `lsmod`) z istniejącym plikiem `.config` i dezaktywuje wszystkie opcje modułowe, które nie odpowiadają żadnemu załadowanemu sterownikowi. Dla każdego nowego symbolu konfiguracyjnego, którego wartość nie jest określona w bazowym pliku, narzędzie wyświetla interaktywny monit - we wszystkich przypadkach zaakceptowano wartości domyślne.

```
make localmodconfig
```

```
root@slack64:/usr/src/linux-7.0.9# make localmodconfig
HOSTCC scripts/basic/fixdep
HOSTCC scripts/kconfig/conf.o
HOSTCC scripts/kconfig/confdata.o
HOSTCC scripts/kconfig/expr.o
LEX scripts/kconfig/lexer.lex.o
YACC scripts/kconfig/parser.tab.[ch]
HOSTCC scripts/kconfig/lexer.lex.o
HOSTCC scripts/kconfig/menu.o
HOSTCC scripts/kconfig/parser.tab.o
HOSTCC scripts/kconfig/preprocess.o
HOSTCC scripts/kconfig/symbol.o
HOSTCC scripts/kconfig/util.o
HOSTLD scripts/kconfig/conf
using config: '.config'
sha1_ssse3 config not found!
sha256_ssse3 config not found!
crt10dif_pclmul config not found!
ledtrig_audio config not found!
crc32_pclmul config not found!
WARNING: FB_SYSMEM_FOPS is required, but nothing in the
current config selects it.
WARNING: FB_SYS_FILLRECT is required, but nothing in the
current config selects it.
WARNING: FB_SYS_IMAGEBLIT is required, but nothing in the
current config selects it.
WARNING: FB_SYS_COPYAREA is required, but nothing in the
current config selects it.
module fb_sys_fops did not have configs CONFIG_FB_SYSMEM_FOPS
.config:860:warning: symbol value '0' invalid for BASE_SHALL
*
* Restart config...
*
*
* Timers subsystem
*
Timer tick handling
1. Periodic timer ticks (constant rate, no dynticks) (HZ_PERIODIC)
> 2. Idle dynticks system (tickless idle) (NO_HZ_IDLE)
3. Full dynticks system (tickless) (NO_HZ_FULL)
choice[1-3]: 2
Old idle dynticks config (NO_HZ) [Y/n/?] y
High Resolution Timer Support (HIGH_RES_TIMERS) [Y/n/?] y
Clocksource watchdog maximum allowable skew (in microseconds) (CLOCKSOURCE_WATCHDOG_MAX_SKEW_US) [125] (NEW) █
```

Przebieg komendy localmodconfig - monity konfiguracyjne

```
# Test find_bit functions (FIND_BIT_BENCHMARK) [N/m/y/?] n
# Test firmware loading via userspace interface (TEST_FIRMWARE) [N/m/y/?] n
# sysctl test driver (TEST_SYSCTL) [N/m/y/?] n
# Compiler context-analysis warnings test (CONTEXT_ANALYSIS_TEST) [N/y/?] (NEW)
# udelay test driver (TEST_UDELAY) [N/m/y/?] n
# Test static keys (TEST_STATIC_KEYS) [N/m/?] n
# kmod stress tester (TEST_KMOD) [N/m/?] n
# module kallsyms find_symbol() test (TEST_KALLSYMS) [N/m/?] (NEW)
# Test memcat_p() helper function (TEST_MEMCAT_P) [N/m/y/?] n
# Test heap/page initialization (TEST_MEMINIT) [N/m/y/?] n
# Test HMM (Heterogeneous Memory Management) (TEST_HMM) [N/m/y/?] n
# Test freeing pages (TEST_FREE_PAGES) [N/m/y/?] n
# Test floating point operations in kernel space (TEST_FPU) [N/m/y/?] n
# Test clocksource watchdog in kernel space (TEST_CLOCKSOURCE_WATCHDOG) [N/m/y/?] n
# Test module for correctness and stress of objpool (TEST_OBJPOOL) [N/m/?] (NEW)
#
# configuration written to .config
#
```

Zakończenie procesu localmodconfig

Następnie rozwiązano wszelkie zależności konfiguracyjne:

```
make olddefconfig
```

```
root@slack64:/usr/src/linux-7.0.9# make olddefconfig
#
# No change to .config
#
```

Rozwiązywanie zależności konfiguracyjnych

3.4. Kompilacja jądra i modułów

Proces kompilacji uruchomiono z wykorzystaniem wszystkich ośmiu dostępnych rdzeni wirtualnych.

```
make -j8 bzImage
```

```
CC      arch/x86/boot/compressed/sev-handle-vc.o
CC      arch/x86/boot/compressed/acpi.o
CC      arch/x86/boot/compressed/mem.o
CC      arch/x86/boot/compressed/efi.o
CC      arch/x86/boot/compressed/misc.o
LZMA     arch/x86/boot/compressed/vmlinux.bin.lzma
HMP1GBV arch/x86/boot/compressed/piggy.s
AS       arch/x86/boot/compressed/piggy.o
LD       arch/x86/boot/compressed/vmlinux
ZOFFSET  arch/x86/boot/offset.h
OBJCOPY  arch/x86/boot/vmlinux.bin
AS       arch/x86/boot/header.o
LD       arch/x86/boot/setup.elf
OBJCOPY  arch/x86/boot/setup.bin
BUILD    arch/x86/boot/bzImage
Kernel: arch/x86/boot/bzImage is ready (#1)
root@slack64:/usr/src/linux-7.0.9#
```

Kompilacja obrazu jądra

```
make -j8 modules
```

```
LD [M]  sound/core/snd.ko
LD [M]  sound/core/snd-hwdep.ko
LD [M]  sound/core/snd-timer.ko
LD [M]  sound/core/snd-pcm.ko
LD [M]  sound/hda/core/snd-hda-core.ko
LD [M]  sound/hda/core/snd-intel-dspcfg.ko
LD [M]  sound/hda/core/snd-intel-sdw-acpi.ko
LD [M]  sound/hda/common/snd-hda-codec.ko
LD [M]  sound/hda/codecs/snd-hda-codec-generic.ko
LD [M]  sound/hda/controllers/snd-hda-intel.ko
LD [M]  net/802/psnap.ko
LD [M]  net/802/stp.ko
LD [M]  net/802/garp.ko
LD [M]  net/802/arp.ko
LD [M]  net/ipv6/ipv6.ko
LD [M]  net/wireless/cfg80211.ko
LD [M]  net/8021q/8021q.ko
LD [M]  net/lld/lld.ko
LD [M]  net/rfkill/rfkill.ko
LD [M]  virt/lib/irqbypass.ko
```

Kompilacja modułów

```
make -j8 modules_install
```

```
INSTALL /lib/modules/7.0.9/kernel/sound/hda/core/snd-intel-sdw-acpi.ko
INSTALL /lib/modules/7.0.9/kernel/sound/hda/codecs/snd-hda-codec-generic.ko
INSTALL /lib/modules/7.0.9/kernel/sound/hda/controllers/snd-hda-intel.ko
INSTALL /lib/modules/7.0.9/kernel/net/802/psnap.ko
INSTALL /lib/modules/7.0.9/kernel/net/802/stp.ko
INSTALL /lib/modules/7.0.9/kernel/net/802/garp.ko
INSTALL /lib/modules/7.0.9/kernel/net/802/arp.ko
INSTALL /lib/modules/7.0.9/kernel/net/ipv6/ipv6.ko
INSTALL /lib/modules/7.0.9/kernel/net/wireless/cfg80211.ko
INSTALL /lib/modules/7.0.9/kernel/net/lld/lld.ko
INSTALL /lib/modules/7.0.9/kernel/net/rfkill/rfkill.ko
INSTALL /lib/modules/7.0.9/kernel/virt/lib/irqbypass.ko
DEPMOD  /lib/modules/7.0.9
```

Instalacja modułów do /lib/modules

```
make -j8 headers_install
```

```
HDRINST usr/include/asm/resource.h
HDRINST usr/include/asm/unistd_64.h
HDRINST usr/include/asm/unistd_x32.h
HDRINST usr/include/asm/poll.h
HDRINST usr/include/asm/ioctls.h
HDRINST usr/include/asm/socket.h
HDRINST usr/include/asm/errno.h
HDRINST usr/include/asm/ioctl.h
HDRINST usr/include/asm/termios.h
HDRINST usr/include/asm/socket.h
HDRINST usr/include/asm/bpf_event.h
HDRINST usr/include/asm/termios.h
HDRINST usr/include/asm/types.h
HDRINST usr/include/asm/unistd_32.h
INSTALL ./usr/include
```

Instalacja nagłówków jądra

3.5. Instalacja jądra i konfiguracja bootloadera

Skompilowane pliki skopiowano do katalogu `/boot` z przyrostkiem identyfikującym zastosowaną metodę.

```
cp arch/x86_64/boot/bzImage /boot/vmlinuz-custom-7.0.9-localmod
cp System.map /boot/System.map-custom-7.0.9-localmod
cp .config /boot/config-custom-7.0.9-localmod
```

```
root@slack64:/usr/src/linux-7.0.9# cp arch/x86_64/boot/bzImage /boot/vmlinuz-custom-7.0.9-localmod
root@slack64:/usr/src/linux-7.0.9# cp System.map /boot/System.map-custom-7.0.9-localmod
root@slack64:/usr/src/linux-7.0.9# cp .config /boot/config-custom-7.0.9-localmod
```

Kopiowanie artefaktów kompilacji do /boot

Wygenerowano obraz początkowy ramdysku (`initrd`) przy użyciu narzędzia `mkinitrd` . Parametry komendy uzyskano z wbudowanego generatora:

```
/usr/share/mkinitrd/mkinitrd_command_generator.sh -k 7.0.9
mkinitrd -c -k 7.0.9 -f ext4 -r /dev/vda1 -m ext4 -u -o /boot/initrd-
custom-7.0.9-localmod.gz
```

```
root@slack64:/usr/src/linux-7.0.9# /usr/share/mkinitrd/mkinitrd_command_generator.sh -k 7.0.9
#
# mkinitrd_command_generator.sh revision 1.45
#
# This script will now make a recommendation about the command to use
# in case you require an initrd image to boot a kernel that does not
# have support for your storage or root filesystem built in
# (such as the Slackware 'generic' kernels').
# A suitable 'mkinitrd' command will be:
mkinitrd -c -k 7.0.9 -f ext4 -r /dev/vda1 -m ext4 -u -o /boot/initrd.gz
```

Przygotowanie komendy mkinitrd

```
# A suitable 'mkinitrd' command will be:
mkinitrd -c -k 7.0.9 -f ext4 -r /dev/vda1 -m ext4 -u -o /boot/initrd.gz
root@slack64:/usr/src/linux-7.0.9# mkinitrd -c -k 7.0.9 -f ext4 -r /dev/vda1 -m ext4 -u -o /boot/initrd-custom-7.0.9-localmod.gz
```

Modyfikacja ścieżki docelowej

```
root@slack64:/usr/src/linux-7.0.9# mkinitrd -c -k 7.0.9 -f ext4 -r /dev/vda1 -m ext4 -u -o /boot/initrd-custom-7.0.9-localmod.gz
49913 blocks
/boot/initrd-custom-7.0.9-localmod.gz created.
Be sure to run lilo again if you use it.
```

Utworzenie obrazu initrd

Zaktualizowano konfigurację bootloadera GRUB, aby uwzględniała nowo skompilowane jądro:

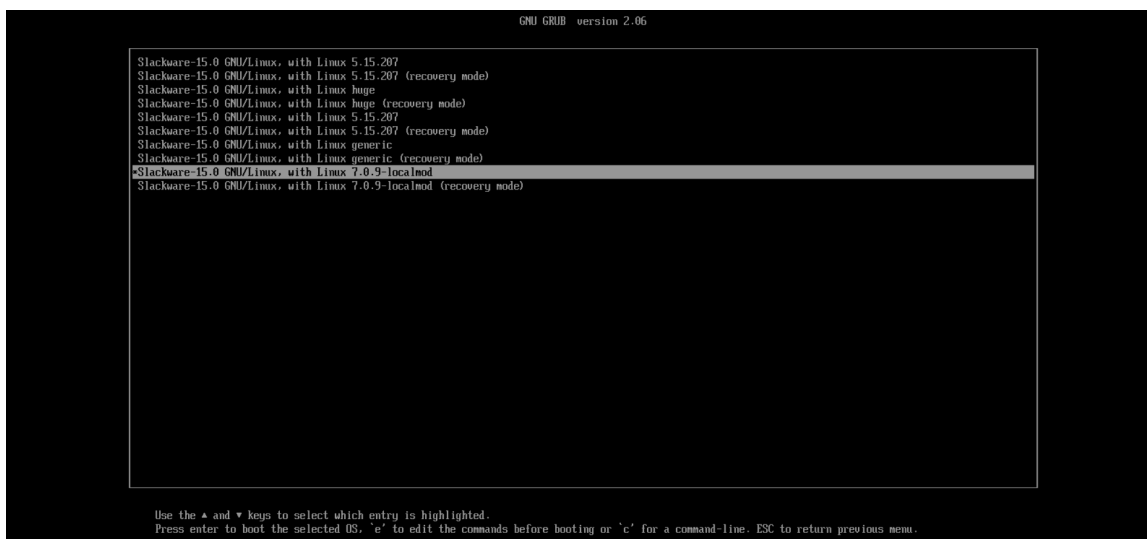
```
grub-mkconfig -o /boot/grub/grub.cfg
```

```
root@slack64:/usr/src/linux-7.0.9# grub-mkconfig -o /boot/grub/grub.cfg
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-huge-5.15.207
Found initrd image: /boot/amd-ucode.img /boot/initrd.gz
Found linux image: /boot/vmlinuz-huge
Found initrd image: /boot/amd-ucode.img /boot/initrd.gz
Found linux image: /boot/vmlinuz-generic-5.15.207
Found initrd image: /boot/amd-ucode.img /boot/initrd.gz
Found linux image: /boot/vmlinuz-generic
Found initrd image: /boot/amd-ucode.img /boot/initrd.gz
Found linux image: /boot/vmlinuz-custom-7.0.9-localmod
Found initrd image: /boot/amd-ucode.img /boot/initrd.gz
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
```

Regeneracja konfiguracji GRUB

3.6. Weryfikacja

Po ponownym uruchomieniu systemu w menu GRUB pojawiła się nowa pozycja odpowiadająca skompilowanemu jądru.



Menu bootloadera GRUB z nowym jądrem

```
uname -r
```

```
root@slack64:~# uname -r
7.0.9
```

Weryfikacja wersji uruchomionego jądra

4. Kompilacja: Metoda 2 (`streamline_config.pl`)

Drugie podejście opierało się na wykorzystaniu skryptu napisanego w języku Perl, dostarczanego w drzewie źródeł jądra (`scripts/kconfig/streamline_config.pl`). Skrypt ten przycina dystrybucyjny plik `.config` na podstawie aktualnie załadowanych modułów sprzętowych (wykrytych przez `lsmod`), analizując przy tym zależności zdefiniowane w plikach `Kconfig` i `Makefile` .

4.1. Czyszczenie środowiska kompilacyjnego

Przed przystąpieniem do drugiej metody środowisko kompilacyjne musiało zostać całkowicie zresetowane. Komenda `make mrproper` usuwa zarówno pliki obiektowe, jak i wszelkie pliki konfiguracyjne, przywracając katalog źródłowy do stanu świeżo rozpakowanego archiwum.

```
cd /usr/src/linux-7.0.9
make mrproper
```

```

root@slack64:~# cd /usr/src/linux-7.0.9
root@slack64:/usr/src/linux-7.0.9# make mrproper
CLEAN arch/x86/boot/compressed
CLEAN arch/x86/boot
CLEAN arch/x86/entry/vdso/vdso32
CLEAN arch/x86/entry/vdso/vdso64
CLEAN arch/x86/kernel/cpu
CLEAN arch/x86/kernel
CLEAN arch/x86/kvm
CLEAN arch/x86/purgatory
CLEAN arch/x86/realmode/rm
CLEAN arch/x86/tools
CLEAN arch/x86/lib
CLEAN certs
CLEAN drivers/firmware/efi/libstub
CLEAN drivers/scsi/aic7xxx
CLEAN drivers/scsi
CLEAN drivers/tty/vt
CLEAN drivers/video/logo
CLEAN fs/unicode
CLEAN init
CLEAN kernel
CLEAN lib/crc
CLEAN lib/raid6
CLEAN lib/test_fortify
CLEAN lib
CLEAN net/wireless
CLEAN usr/include
CLEAN usr
CLEAN .
CLEAN vmlinux.symvers modules.builtin modules.builtin.modinfo vmlinux.unstripped vmlinux.export.c
CLEAN scripts/basic
CLEAN scripts/gcc-plugins
CLEAN scripts/kconfig
CLEAN scripts/mod
CLEAN scripts
CLEAN include/config include/generated arch/x86/include/generated .config .config.old .version Module.symvers

```

Czyszczenie katalogu źródeł

Przywrócono bazowy plik konfiguracyjny z archiwum systemowego:

```
zcat /proc/config.gz > .config
```

```
root@slack64:/usr/src/linux-7.0.9# zcat /proc/config.gz > .config
```

Odzyskiwanie konfiguracji bazowej

4.2. Generowanie konfiguracji skryptem Perl

Skrypt `streamline_config.pl` odczytuje bieżącą listę modułów i generuje zoptymalizowany plik konfiguracyjny:

```
perl scripts/kconfig/streamline_config.pl > config_newmethod
mv config_newmethod .config
```

```

root@slack64:/usr/src/linux-7.0.9# perl scripts/kconfig/streamline_config.pl > config_newmethod
using config: '.config'
sha1_ssse3 config not found!
crc32_pclmul config not found!
ledtrig_audio config not found!
sha256_ssse3 config not found!
crt18dif_pclmul config not found!
module fb_sys_fops did not have configs CONFIG_FB_SYSMEM_FOPS
root@slack64:/usr/src/linux-7.0.9# mv config_newmethod .config

```

Wykonanie skryptu `streamline_config.pl`

Rozwiązano zależności konfiguracyjne:

```
make olddefconfig
```

```

root@slack64:/usr/src/linux-7.0.9# make olddefconfig
HOSTCC scripts/basic/fixdep
HOSTCC scripts/kconfig/conf.o
HOSTCC scripts/kconfig/confdata.o
HOSTCC scripts/kconfig/expr.o
LEX scripts/kconfig/lexer.lex.c
YACC scripts/kconfig/parser.tab.[ch]
HOSTCC scripts/kconfig/lexer.lex.o
HOSTCC scripts/kconfig/menu.o
HOSTCC scripts/kconfig/parser.tab.o
HOSTCC scripts/kconfig/preprocess.o
HOSTCC scripts/kconfig/symbol.o
HOSTCC scripts/kconfig/util.o
HOSTLD scripts/kconfig/conf
.config:849:warning: symbol value '0' invalid for BASE_SMALL
#
# configuration written to .config
#

```

Rozwiązywanie zależności konfiguracyjnych

4.3. Kompilacja jądra i modułów

Analogicznie do pierwszej metody, kompilację przeprowadzono z wykorzystaniem ośmiu rdzeni wirtualnych.

```
make -j8 bzImage
```

```
CC arch/x86/boot/compressed/sev-handle-vc.o
CC arch/x86/boot/compressed/acpi.o
CC arch/x86/boot/compressed/mem.o
CC arch/x86/boot/compressed/efi.o
CC arch/x86/boot/compressed/misc.o
LZMA arch/x86/boot/compressed/vmlinux.bin.lzma
MKPIGgy arch/x86/boot/compressed/piggy.S
AS arch/x86/boot/compressed/piggy.o
LD arch/x86/boot/compressed/vmlinux
ZOFFSET arch/x86/boot/zoffset.h
OBJCOPY arch/x86/boot/vmlinux.bin
AS arch/x86/boot/header.o
LD arch/x86/boot/setup.elf
OBJCOPY arch/x86/boot/setup.bin
BUILD arch/x86/boot/bzImage
Kernel: arch/x86/boot/bzImage is ready (#1)
```

Kompilacja obrazu jądra

```
make -j8 modules
```

```
LD [M] sound/hda/core/snd-intel-dspcfg.ko
LD [M] sound/hda/core/snd-intel-sdw-acpi.ko
LD [M] sound/hda/common/snd-hda-codec.ko
LD [M] sound/hda/codecs/snd-hda-codec-generic.ko
LD [M] sound/hda/controllers/snd-hda-intel.ko
LD [M] net/802/psnap.ko
LD [M] net/802/stp.ko
LD [M] net/802/garp.ko
LD [M] net/ipv6/ipv6.ko
LD [M] net/802/mrp.ko
LD [M] net/8021q/8021q.ko
LD [M] net/rfkill/rfkill.ko
LD [M] net/wireless/cfg80211.ko
LD [M] net/llc/llc.ko
LD [M] virt/lib/irqbypass.ko
```

Kompilacja modułów

```
make -j8 modules_install
```

```
INSTALL /lib/modules/7.0.9/kernel/sound/hda/core/snd-intel-sdw-acpi.ko
INSTALL /lib/modules/7.0.9/kernel/sound/hda/common/snd-hda-codec.ko
INSTALL /lib/modules/7.0.9/kernel/sound/hda/controllers/snd-hda-intel.ko
INSTALL /lib/modules/7.0.9/kernel/sound/hda/codecs/snd-hda-codec-generic.ko
INSTALL /lib/modules/7.0.9/kernel/net/802/psnap.ko
INSTALL /lib/modules/7.0.9/kernel/net/802/stp.ko
INSTALL /lib/modules/7.0.9/kernel/net/802/garp.ko
INSTALL /lib/modules/7.0.9/kernel/net/802/mrp.ko
INSTALL /lib/modules/7.0.9/kernel/net/ipv6/ipv6.ko
INSTALL /lib/modules/7.0.9/kernel/net/8021q/8021q.ko
INSTALL /lib/modules/7.0.9/kernel/net/wireless/cfg80211.ko
INSTALL /lib/modules/7.0.9/kernel/net/llc/llc.ko
INSTALL /lib/modules/7.0.9/kernel/virt/lib/irqbypass.ko
INSTALL /lib/modules/7.0.9/kernel/net/rfkill/rfkill.ko
DEPMOD /lib/modules/7.0.9
```

Instalacja modułów do /lib/modules

```
make -j8 headers_install
```

```
HDRINST usr/include/asm/resource.h
HDRINST usr/include/asm/unistd_64.h
HDRINST usr/include/asm/unistd_x32.h
HDRINST usr/include/asm/poll.h
HDRINST usr/include/asm/ioctls.h
HDRINST usr/include/asm/socbits.h
HDRINST usr/include/asm/errno.h
HDRINST usr/include/asm/termbits.h
HDRINST usr/include/asm/ioctl.h
HDRINST usr/include/asm/socket.h
HDRINST usr/include/asm/bpf_perf_event.h
HDRINST usr/include/asm/termios.h
HDRINST usr/include/asm/unistd_32.h
HDRINST usr/include/asm/types.h
INSTALL ./usr/include
```

Instalacja nagłóweków jądra

4.4. Instalacja jądra i konfiguracja bootloadera

Artefakty kompilacji skopiowano do katalogu /boot z odrębnym przyrostkiem:

```
cp arch/x86_64/boot/bzImage /boot/vmlinux-custom-7.0.9-streamline
cp System.map /boot/System.map-custom-7.0.9-streamline
cp .config /boot/config-custom-7.0.9-streamline
```

```
root@slack64:/usr/src/linux-7.0.9# cp arch/x86_64/boot/bzImage /boot/vmlinuz-custom-7.0.9-streamline
root@slack64:/usr/src/linux-7.0.9# cp System.map /boot/System.map-custom-7.0.9-streamline
root@slack64:/usr/src/linux-7.0.9# cp .config /boot/config-custom-7.0.9-streamline
```

Kopiowanie artefaktów kompilacji do /boot

Wygenerowano obraz ramdysku:

```
mkinitrd -c -k 7.0.9 -f ext4 -r /dev/vda1 -m ext4 -u -o /boot/initrd-
custom-7.0.9-streamline.gz
```

```
root@slack64:/usr/src/linux-7.0.9# mkinitrd -c -k 7.0.9 -f ext4 -r /dev/vda1 -m ext4 -u -o /boot/initrd-custom-7.0.9-streamline.gz
49913 blocks
/boot/initrd-custom-7.0.9-streamline.gz created.
Be sure to run lilo again if you use it.
```

Generowanie initrd dla metody 2

Zaktualizowano konfigurację bootloadera:

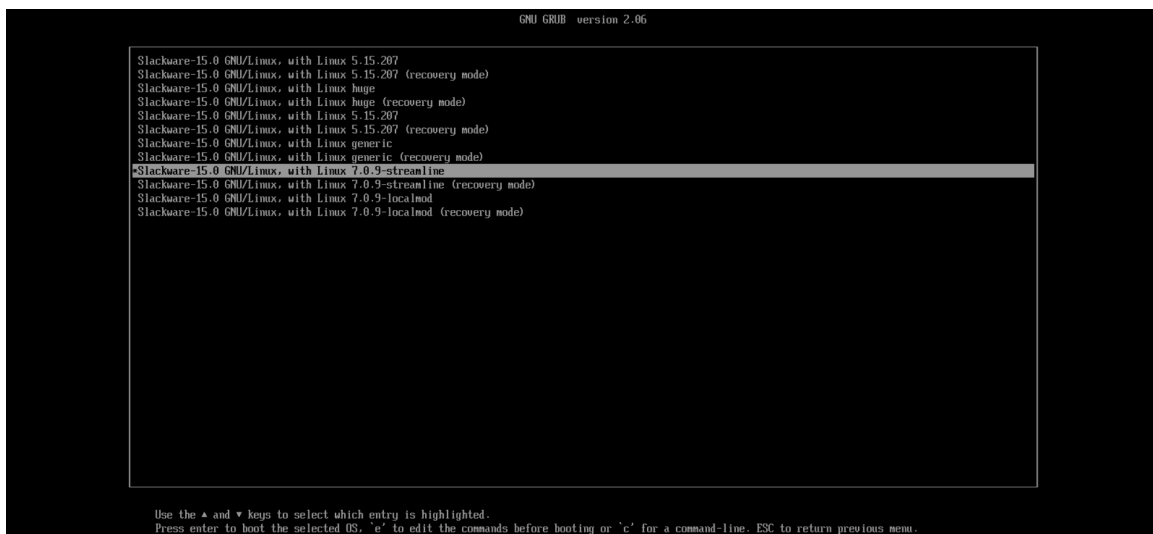
```
grub-mkconfig -o /boot/grub/grub.cfg
```

```
root@slack64:/usr/src/linux-7.0.9# grub-mkconfig -o /boot/grub/grub.cfg
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-huge-5.15.207
Found initrd image: /boot/amd-ucode.img /boot/initrd.gz
Found linux image: /boot/vmlinuz-huge
Found initrd image: /boot/amd-ucode.img /boot/initrd.gz
Found linux image: /boot/vmlinuz-generic-5.15.207
Found initrd image: /boot/amd-ucode.img /boot/initrd.gz
Found linux image: /boot/vmlinuz-generic
Found initrd image: /boot/amd-ucode.img /boot/initrd.gz
Found linux image: /boot/vmlinuz-custom-7.0.9-streamline
Found initrd image: /boot/amd-ucode.img /boot/initrd.gz
Found linux image: /boot/vmlinuz-custom-7.0.9-localmod
Found initrd image: /boot/amd-ucode.img /boot/initrd.gz
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
```

Regeneracja konfiguracji GRUB

4.5. Weryfikacja

Po ponownym uruchomieniu wybrano jądro skompilowane metodą `streamline_config.pl` z poziomu menu zaawansowanego GRUB.



Menu bootloadera GRUB z oboma jądrami

```
uname -r
```

```
root@slack64:~# uname -r  
7.0.9
```

Weryfikacja wersji uruchomionego jądra

5. Archiwizacja i transfer danych wynikowych

Po zakończeniu obu kompilacji przygotowano archiwum źródłowe zawierające kod jądra wraz z oboma plikami konfiguracyjnymi. Pliki `.config` wygenerowane przez każdą z metod zostały wcześniej zachowane w katalogu `/boot` - skopiowano je z powrotem do drzewa źródeł pod unikalnymi nazwami.

```
cp /boot/config-custom-7.0.9-localmod /usr/src/linux-  
7.0.9/.config.localmod  
cp /boot/config-custom-7.0.9-streamline /usr/src/linux-  
7.0.9/.config.streamline
```

```
root@slack64:~# cp /boot/config-custom-7.0.9-localmod /usr/src/linux-7.0.9/.config.localmod  
root@slack64:~# cp /boot/config-custom-7.0.9-streamline /usr/src/linux-7.0.9/.config.streamline
```

Kopiowanie obu konfiguracji do katalogu źródeł

Usunięto pliki obiektowe powstałe w procesie kompilacji, zachowując kod źródłowy i pliki konfiguracyjne:

```
cd /usr/src/linux-7.0.9  
make clean
```

```
root@slack64:~# cd /usr/src/linux-7.0.9  
root@slack64:/usr/src/linux-7.0.9# make clean  
CLEAN arch/x86/boot/compressed  
CLEAN arch/x86/boot  
CLEAN arch/x86/entry/vdso/vdso32  
CLEAN arch/x86/entry/vdso/vdso64  
CLEAN arch/x86/kernel/cpu  
CLEAN arch/x86/kernel  
CLEAN arch/x86/kvm  
CLEAN arch/x86/purgatory  
CLEAN arch/x86/realmode/rm  
CLEAN arch/x86/tools  
CLEAN arch/x86/lib  
CLEAN certs  
CLEAN drivers/firmware/efi/libstub  
CLEAN drivers/scsi/aic7xxx  
CLEAN drivers/scsi  
CLEAN drivers/tty/vt  
CLEAN drivers/video/logo  
CLEAN fs/unicode  
CLEAN init  
CLEAN kernel  
CLEAN lib/crc  
CLEAN lib/raid6  
CLEAN lib/testfortify  
CLEAN lib  
CLEAN net/wireless  
CLEAN usr/include  
CLEAN usr  
CLEAN .  
CLEAN vmlinux.symvers modules.builtin modules.builtin.modinfo vmlinux.unstripped vmlinux.export.c
```

Czyszczenie plików obiektowych

Utworzono skompresowane archiwum:

```
cd /usr/src  
tar -czvf compiled-kernel.tar.gz linux-7.0.9/
```

```
linux-7.0.9/samples/vfs/test-fsmount.c
linux-7.0.9/samples/vfs/Makefile
linux-7.0.9/samples/vfs/test-statx.c
linux-7.0.9/samples/vfs/test-list-all-mounts.c
linux-7.0.9/samples/vfs/mountinfo.c
linux-7.0.9/samples/vfs/.gitignore
linux-7.0.9/samples/vfs/samples-vfs.h
linux-7.0.9/samples/landlock/
linux-7.0.9/samples/landlock/Makefile
linux-7.0.9/samples/landlock/sandboxer.c
linux-7.0.9/samples/landlock/.gitignore
linux-7.0.9/samples/hidraw/
linux-7.0.9/samples/hidraw/Makefile
linux-7.0.9/samples/hidraw/hid-example.c
linux-7.0.9/samples/hidraw/.gitignore
```

Tworzenie archiwum tar.gz

Archiwum przetransferowano na system nadrzędny za pomocą protokołu SFTP:

```
sftp root@192.168.122.184
get /usr/src/compiled-kernel.tar.gz
```

```
} sftp root@192.168.122.184
(root@192.168.122.184) Password:
Connected to 192.168.122.184.
sftp> get /usr/src/compiled-kernel.tar.gz
Fetching /usr/src/compiled-kernel.tar.gz to compiled-kernel.tar.gz
compiled-kernel.tar.gz                                     100% 255MB 470.4MB/s 00:00
```

Transfer archiwum przez SFTP

6. Wnioski

Obie zastosowane metody - `make localmodconfig` oraz skrypt `streamline_config.pl` - realizują ten sam cel: redukcję konfiguracji jądra do podzbioru modułów faktycznie wymaganych przez bieżący sprzęt. Różnią się natomiast sposobem działania i zastosowaniem.

`make localmodconfig` jest wbudowanym celem systemu budowania jądra (Kbuild). Odczytuje listę załadowanych modułów z `lsmod` i interaktywnie pyta użytkownika o wartości nowych, nieznanych symboli konfiguracyjnych. Ta interaktywność sprawia, że metoda jest intuicyjna w użyciu ręcznym, ale utrudnia jej zastosowanie w zautomatyzowanych potokach CI/CD bez dodatkowych narzędzi (np. `yes "" |`).

`streamline_config.pl` operuje na poziomie plików `Kconfig` i `Makefile`, analizując zależności między opcjami konfiguracyjnymi. Wynik działania skryptu jest przekierowywany do pliku, co czyni go w pełni nieinteraktywnym i predestynowanym do użycia w skryptach automatyzujących proces budowania.

Z perspektywy bezpieczeństwa, oba podejścia zmniejszają powierzchnię ataku jądra systemowego. Usunięcie nieużywanych sterowników i podsystemów eliminuje potencjalne wektory ataków w kodzie, który nigdy nie byłby wykorzystywany na danej maszynie. Z perspektywy wydajnościowej, ograniczenie liczby kompilowanych modułów skraca czas kompilacji i zmniejsza rozmiar wynikowego obrazu jądra oraz drzewa modułów w `/lib/modules`.

Sam proces kompilacji w obu przypadkach przebiegł bezproblemowo - nie odnotowano błędów kompilacji, brakujących zależności ani trudności z integracją nowego jądra z bootloaderem GRUB. Czas kompilacji nie był mierzony, ponieważ nie stanowił przedmiotu

analizy w ramach tego zadania, a oba przebiegi zakończyły się w rozsądnym czasie na przydzielonych ośmiu rdzeniach wirtualnych.

Gdyby konieczne było wskazanie preferowanej metody, wybór padłby na `streamline_config.pl`. Jej w pełni nieinteraktywny charakter (brak monitów wymagających ręcznej akceptacji) czyni ją znacznie lepiej przystosowaną do wykorzystania w środowiskach zautomatyzowanych, takich jak potoki CI/CD czy skrypty budujące niestandardowe obrazy systemowe.